

TOMLEE HOMECARE LLC

COMPLETE MASTER BUILD PROMPT

Production Website + Lead Qualification Platform

Next.js + Supabase + TypeScript

Development Environment: Google Antigravity

1. PROJECT ROLE

You are the lead:

- Senior Full-Stack Engineer
- Product Architect
- UX/UI Designer
- Conversion Rate Optimization Specialist
- SEO Engineer
- Accessibility Engineer
- Security Engineer
- Database Architect
- Technical Documentation Engineer

Your task is to design and build the complete production-ready website and lead-generation platform for:

TomLee Homecare LLC

TomLee Homecare LLC is a Georgia-based non-medical home care agency.

The business currently focuses on **private-pay home care**.

Medicaid and waiver services are future goals and must be architected into the system but must NOT be presented as currently available unless explicitly activated by an authorized administrator after client approval.

The final product must look and behave like a professional, established U.S. home-care company.

This is NOT a generic template website.

It is a:

2. PROJECT OBJECTIVES

The application must accomplish the following:

1. Build a professional and trustworthy TomLee Homecare brand presence.
 2. Communicate compassion, reliability, dignity and peace of mind.
 3. Clearly communicate current private-pay services.
 4. Position TomLee as an attractive alternative to unreliable home-care providers.
 5. Generate high-quality care inquiries.
 6. Qualify leads before they reach the business.
 7. Capture whether a visitor is unhappy with their current provider.
 8. Capture care type and payment method.
 9. Score leads using configurable rules.
 10. Route qualified leads appropriately.
 11. Provide a content/resource center.
 12. Provide an admin dashboard.
 13. Allow administrators to manage services without code changes.
 14. Allow future Medicaid and waiver services to be activated later without rebuilding the application.
 15. Be highly responsive.
 16. Be SEO-ready.
 17. Be accessible.
 18. Be secure.
 19. Minimize sensitive information collection.
 20. Provide a maintainable codebase suitable for professional handover.
-

3. ORIGINAL CLIENT SCOPE

The website scope includes:

Brand Identity

- Refine existing logo
- Primary logo
- Secondary logo
- Favicon
- Brand colors
- Typography
- Brand voice

Domain / Hosting

- Domain setup
- DNS
- SSL
- Hosting
- Custom organization email

Website

Core pages:

- Home
- About Us
- Services
- Resources
- Contact

Additional required pages:

- Request Care
- Service Detail
- Resource Detail
- Privacy Policy
- Terms of Use
- 404

Lead Qualification

- Contact/inquiry forms
- Care type
- Payment method
- Service requested
- Location
- Lead scoring
- Lead qualification
- Lead routing
- Notifications

Compliance

- Privacy Policy
- Terms
- HIPAA-aware architecture
- Secure data handling

Handover

- Source code
 - Admin access
 - Brand assets
 - Documentation
 - Management instructions
-

4. BRAND INFORMATION

The supplied TomLee Home Care logo is the primary visual reference.

The existing logo contains:

- Teal/turquoise healthcare icon
- Lime/green accent
- Dark charcoal/gray wordmark
- Human/care symbol integrated with a healthcare/cross element

Do NOT radically redesign the logo.

Refine it only where necessary to improve:

- spacing
- proportions
- sharpness
- consistency
- digital usability
- favicon compatibility
- responsive usage

Create:

1. Primary logo
2. Horizontal logo
3. Compact logo/icon
4. Favicon
5. Light-background version
6. Dark-background version

Preserve brand recognition.

5. BRAND COLOR DIRECTION

Use the existing logo as the source of truth for the initial palette.

The design system should revolve around:

Primary

Teal / turquoise healthcare tone.

Accent

Lime/green.

Neutral

Dark charcoal / slate.

Background

Warm white / off-white.

Supporting colors

Soft muted versions of the brand colors.

Do NOT use excessive bright green.

The lime accent should be used strategically for:

- highlights
- icons
- small accents
- success indicators
- selected states

The website should feel:

- warm
- premium
- calm
- trustworthy
- modern
- human
- professional

Avoid an overly clinical hospital aesthetic.

6. BRAND POSITIONING

The central positioning should revolve around:

Reliable Caregivers. Peace of Mind for Families.

The brand should communicate that families are not simply buying hours of caregiving.

They are buying:

- reliability
 - trust
 - consistency
 - communication
 - dignity
 - peace of mind
-

7. COMPETITIVE POSITIONING

A major marketing angle is helping people who are unhappy with their current home-care provider.

The website must include a prominent conversion section around:

NOT HAPPY WITH YOUR CURRENT HOME CARE PROVIDER?

Supporting message:

Switching Is Easier Than You Think.

Do not make this aggressive or insulting toward competitors.

Instead, focus on common frustrations:

- unreliable caregivers
- last-minute cancellations
- poor communication
- lack of caregiver consistency
- insufficient family updates
- care needs not being properly addressed
- difficulty getting flexible care

Position TomLee as the easier, more dependable alternative.

8. IMPORTANT CLAIM VERIFICATION RULE

The client has discussed the following potential differentiators:

- Free In-Home Care Assessment
- Same-Day / Next-Day Care Start
- Caregiver Matching
- Backup Caregiver Guarantee / No-Call-No-Show Protection
- Family Updates After Visits
- Free Home Fall-Safety Check
- CNA/CMA-led care with 5+ years of senior-care experience
- Flexible care options
- No Long-Term Contracts
- Easy Switching From Your Current Home Care Agency

These are CLIENT-PROVIDED / PROPOSED CLAIMS.

Do NOT silently treat them as verified facts.

Build the website architecture around them, but mark them internally as:

CLIENT APPROVAL REQUIRED

before final publication.

Do not invent additional claims.

Never invent:

- licenses
 - certifications
 - awards
 - years in business
 - staff credentials
 - testimonials
 - ratings
 - number of clients
 - locations
 - response times
 - insurance participation
 - Medicaid participation
 - waiver participation
-

9. SERVICES

The following services have been discussed and should be represented in the architecture:

Personal Care

- Bathing
- Grooming
- Dressing
- Personal hygiene assistance

Companion Care

- Conversation
- Social engagement
- Companionship
- Assistance with daily routines

Meal Preparation

- Meal preparation
- Meal assistance
- Kitchen support

Light Housekeeping

- Light household tasks
- Tidying
- Laundry assistance where approved

Medication Reminders

Reminder support only.

Do not imply medication administration unless the client explicitly confirms that the business is licensed and authorized to provide it.

Mobility & Transfers

- Mobility support
- Transfers
- Safe movement assistance

Toileting / Incontinence Care

Compassionate personal-care support.

Dementia / Alzheimer's Support

Non-medical support for individuals and families where this service is approved.

Respite Care

Temporary support for family caregivers.

Overnight / Weekend Care

Flexible support outside standard daytime hours.

10. SERVICE ARCHITECTURE

Services MUST be database-driven.

Create a Supabase `services` table.

Recommended fields:

```
id
name
slug
short_description
description
category
status
is_featured
is_private_pay
is_medicaid
is_waiver
image_url
sort_order
seo_title
seo_description
created_at
updated_at
```

Status values:

```
draft
active
```

```
future
archived
```

Public behavior:

```
active      → publicly visible
draft       → admin only
future      → admin only unless explicitly enabled
archived    → hidden
```

11. FUTURE MEDICAID / WAIVER ARCHITECTURE

The system must be future-ready.

Create service/payment categories that can eventually support:

- Private Pay
- Medicaid
- Waiver
- Other

Currently:

```
Private Pay = ACTIVE
Medicaid = FUTURE
Waiver = FUTURE
```

Do not display Medicaid or waiver as current offerings.

An authorized administrator must eventually be able to:

- activate a service
- deactivate a service
- change category
- change payment eligibility
- mark a service as future
- archive a service

No developer should be required to edit frontend code to activate a future service.

12. WEBSITE INFORMATION ARCHITECTURE

Create the following routes:

```
/
/about
/services
/services/[slug]
/resources
/resources/[slug]
/contact
/request-care
/privacy
/terms
/404
```

Protected admin:

```
/admin
/admin/leads
/admin/services
/admin/resources
/admin/faqs
/admin/qualification-rules
/admin/settings
/admin/users
```

13. HOMEPAGE

The homepage must be conversion-focused.

Do not simply build a standard brochure website.

Structure:

SECTION 1 — Header

Navigation:

- Home
- About

- Services
- Resources
- Contact

Primary CTA:

Request Care

Optional secondary CTA:

Call Us

Header should be sticky or intelligently compact on scroll.

Mobile navigation must be accessible.

14. HERO SECTION

Create a premium hero section using warm, realistic home-care photography.

Visual direction:

- Caregiver helping an older adult
- Natural home environment
- Genuine interaction
- Warm lighting
- Diverse representation
- Avoid hospital imagery

Primary headline direction:

Reliable Home Care. Peace of Mind for Your Family.

Supporting copy:

Compassionate, dependable in-home care designed around your loved one's needs, preferences and daily routine.

Primary CTA:

Get a Free Care Assessment

Secondary CTA:

Explore Our Services

Do not invent specific claims.

15. HERO TRUST STRIP

Under the hero, include concise trust/value indicators.

Examples:

- Personalized Care
- Reliable Caregivers
- Flexible Care Options
- Private-Pay Home Care

Only use claims approved by the client.

16. SWITCHING SECTION

This should be one of the strongest homepage sections.

Headline:

Not Happy With Your Current Home Care Provider?

Subheadline:

Switching Is Easier Than You Think.

Explain common reasons families consider changing agencies.

Use cards or icons:

- Caregiver reliability
- Communication
- Last-minute cancellations
- Consistency
- Flexibility
- Family updates

CTA:

Talk to TomLee

or:

Explore Your Care Options

17. WHY CHOOSE TOMLEE

Create a visually strong differentiator section.

Potential cards:

Free In-Home Care Assessment

Caregiver Matching

Backup Caregiver Protection

Family Updates After Visits

Free Home Fall-Safety Check

Flexible Care Options

No Long-Term Contracts

Easy Switching

Only publish items after client approval.

18. SERVICES SECTION

Display active services from Supabase.

Use premium service cards.

Each card:

- image
- service name
- short description
- learn more CTA

Include:

View All Services

Do not hard-code service cards into the page.

19. CARE EXPERIENCE SECTION

Create a section communicating the TomLee care philosophy.

Focus on:

- Dignity
- Respect
- Reliability
- Personal attention
- Family communication

Avoid clinical language.

20. HOW IT WORKS

Create a simple process:

01

Tell Us What You Need

02

Complete Your Care Assessment

03

We Help Match Your Care Needs

04

Start Care

The exact wording may be adjusted after client review.

21. WHO WE SERVE

Create a warm section describing appropriate home-care clients.

Do not make medical claims.

Potential groups:

- Older adults
 - Seniors needing daily support
 - Individuals needing companionship
 - Families needing respite
 - Individuals needing assistance with personal care
 - Families transitioning from another home-care agency
-

22. PRIVATE-PAY SECTION

Clearly communicate the current business model:

Private-Pay Home Care

Explain that clients/families currently arrange and pay privately for services.

Do not state Medicaid or waiver participation.

Future program information should only appear in admin or future-content configuration.

23. TESTIMONIALS

Do not invent testimonials.

Build a reusable testimonial component but only display it when approved client testimonials exist.

Database-driven testimonial architecture may be included for future use.

24. RESOURCES SECTION

Display recent resources from Supabase.

Include:

- resource image
- title
- excerpt
- category
- date
- read more

CTA:

Visit Resource Center

25. FINAL HOMEPAGE CTA

Create a strong closing section:

Ready for a Better Home Care Experience?

Supporting copy:

Whether you're arranging care for the first time or considering a change from your current provider, we're here to help you understand your options.

CTA:

Get Your Free Care Assessment

26. ABOUT PAGE

Build:

Hero

Company Story

Mission

Values

Care Philosophy

Why Families Choose TomLee

CTA

Do not invent company history.

Use placeholders for information not yet supplied.

27. SERVICES PAGE

Create a service directory.

Include:

- Page hero
- Intro
- Service categories
- Active service cards
- CTA
- FAQ
- Contact/request care section

Only active services appear publicly.

28. SERVICE DETAIL PAGE

Dynamic route:

/services/[slug]

Include:

- Breadcrumb
- Service title
- Hero image
- Overview
- What the service includes
- Who it is for
- Benefits
- Related services
- FAQs
- CTA

CTA:

Request Care

29. RESOURCES

Create a full resource center.

Categories:

- Home Care
- Family Resources
- Caregiving
- Private Pay
- Guides
- Dementia Care
- Senior Safety
- FAQs

Admin must be able to create and manage resources.

30. FAQ SYSTEM

Create database-driven FAQs.

Fields:

```
id
question
answer
category
status
sort_order
created_at
updated_at
```

Admin CRUD required.

Use accessible accordion behavior.

31. CONTACT PAGE

Include:

- Business contact information
- Phone
- Email
- Service area
- Contact form
- Request Care CTA

Do not invent contact details.

32. REQUEST CARE PAGE

This is a CORE business feature.

Do not create a basic:

Name / Email / Message

form.

Create a multi-step care inquiry and qualification experience.

33. REQUEST CARE STEP 1

Question:

What brings you to TomLee Homecare?

Options:

I'm looking for home care for the first time
I'm unhappy with my current home care provider
I'm looking for additional care
I'm comparing home care providers
I'm looking for temporary/respite care
I'm looking for information
Other

Store as:

lead_intent

This field is extremely important for marketing intelligence.

34. REQUEST CARE STEP 2

Ask:

Who needs care?

Options may include:

- Myself
- Parent
- Spouse/partner
- Grandparent
- Other family member
- Other

Store:

care_recipient_relationship

35. REQUEST CARE STEP 3

Ask:

What type of help are you looking for?

Allow multiple selections.

Options:

- Personal Care
- Bathing/Grooming/Dressing
- Companionship
- Meal Preparation
- Light Housekeeping

- Medication Reminders
- Mobility/Transfers
- Toileting/Incontinence Care
- Dementia/Alzheimer's Support
- Respite Care
- Overnight Care
- Weekend Care
- Other

Store:

```
services_requested
```

36. REQUEST CARE STEP 4

Ask:

Where is care needed?

Collect only the location information necessary for service-area qualification.

Prefer:

- City
- State
- ZIP code

Avoid collecting a detailed home address unless operationally necessary.

Store:

```
city  
state  
zip_code  
service_area_status
```

37. REQUEST CARE STEP 5

Ask:

How do you plan to pay for care?

Options:

Private Pay
Medicaid
Waiver
Not Sure
Other

Current business logic:

Private Pay → Current offering

Medicaid → Future / Review

Waiver → Future / Review

Do not promise service availability.

38. REQUEST CARE STEP 6

Ask:

When do you need care?

Options:

Immediately
Within a few days
Within 1-2 weeks
Within a month
I'm planning ahead
Not sure

Store:

urgency

39. REQUEST CARE STEP 7

Collect:

- First name
- Last name
- Email
- Phone
- Preferred contact method

Optional:

- Best time to contact

Do not collect unnecessary medical information.

40. PRIVACY WARNING

Before submission display:

Please do not include medical records, diagnoses, Social Security numbers, insurance identification numbers, or other sensitive medical information in this form.

Only collect information necessary for the inquiry and qualification process.

41. LEAD DATABASE

Create:

leads

Suggested structure:

```
id
first_name
last_name
email
phone
preferred_contact_method
lead_intent
```



```
care_recipient_relationship
services_requested
city
state
zip_code
service_area_status
payment_method
urgency
qualification_score
qualification_status
routing_status
source
consent
created_at
updated_at
```

Avoid storing unnecessary PHI.

42. QUALIFICATION ENGINE

Do not hard-code qualification logic inside UI components.

Create a configurable engine.

Create:

```
qualification_rules
```

Fields:

```
id
name
field
operator
value
score
result_status
active
created_at
updated_at
```

43. EXAMPLE QUALIFICATION RULES

These are initial examples only.

Payment

Private Pay:

+50

Approved Service

+30

Supported Service Area

+20

Urgent Need

+10

Complete Contact Information

+10

44. QUALIFICATION STATUS

Initial example:

80+ = QUALIFIED

50-79 = NEEDS_REVIEW

Below 50 = NOT_CURRENTLY_QUALIFIED

Make these thresholds configurable.

The business must be able to change qualification logic later.

45. SPECIAL SWITCHING LEAD LOGIC

If:

```
lead_intent = unhappy_with_current_provider
```

capture this prominently in the admin lead profile.

Potential lead badge:

SWITCHING PROVIDER

This should allow TomLee to identify prospects who are already receiving care but are actively looking for another provider.

This is a valuable high-intent lead category.

46. LEAD ROUTING

Implement:

QUALIFIED

→ primary business notification

NEEDS_REVIEW

→ internal review queue

NOT_CURRENTLY_QUALIFIED

→ retain only according to approved data-retention policy

Do not send unnecessary sensitive information through email.

47. ADMIN DASHBOARD

Create:

/admin

Dashboard metrics:

- Total Leads
- Qualified Leads
- Needs Review
- Switching Leads
- Leads This Week
- Leads This Month
- Active Services
- Published Resources

Keep dashboard simple.

48. ADMIN LEADS

Create:

/admin/leads

Features:

- Search
- Filter
- Sort
- Date range
- Qualification status
- Lead intent
- Payment method
- Service
- Location
- Lead details
- Internal notes
- Status updates

Use badges:

QUALIFIED

NEEDS REVIEW

NOT CURRENTLY QUALIFIED

SWITCHING PROVIDER

49. ADMIN SERVICES

Create:

/admin/services

Admin can:

- Create
 - Edit
 - Publish
 - Unpublish
 - Archive
 - Activate
 - Deactivate
 - Mark future
 - Feature
 - Set category
 - Set payment eligibility
 - Set SEO metadata
 - Set image
-

50. ADMIN RESOURCES

Create:

/admin/resources

Features:

- Create
- Edit

- Draft
- Publish
- Unpublish
- Archive
- Delete
- Category
- Image
- SEO metadata

51. ADMIN FAQ

Create:

```
/admin/faqs
```

CRUD functionality.

Allow:

- Reordering
- Publish/unpublish
- Category

52. ADMIN QUALIFICATION RULES

Create:

```
/admin/qualification-rules
```

Admin can:

- Create rule
- Edit rule
- Enable/disable
- Set field
- Set operator
- Set value
- Set score
- Set result

Do not allow arbitrary code execution.

53. ADMIN SETTINGS

Create:

/admin/settings

Settings:

- Business name
- Phone
- Email
- Service area
- Notification email
- Social links
- Logo
- Primary CTA
- Default SEO
- Website status

Do not hard-code ordinary business information.

54. AUTHENTICATION

Use Supabase Auth.

Implement:

- Login
- Logout
- Password reset
- Session management
- Protected admin routes

Roles:

ADMIN
EDITOR

ADMIN:

Full access.

EDITOR:

Content management but restricted access to sensitive lead/security configuration.

Do not rely only on frontend route hiding.

Enforce authorization server-side and through Supabase RLS.

55. SUPABASE DATABASE

Use PostgreSQL.

Tables should include at minimum:

```
admin_users
services
resources
faqs
leads
qualification_rules
site_settings
```

Optional future tables:

```
testimonials
service_categories
lead_notes
audit_logs
```

Use proper foreign keys and timestamps.

56. ROW LEVEL SECURITY

Enable RLS on all relevant tables.

Public users must NOT have unrestricted access to:

- leads
- qualification rules
- admin users

- site administration
- sensitive settings

Use role-based policies.

Never expose the Supabase service-role key to the browser.

57. SECURITY

Implement:

- Server-side validation
- Client-side validation
- RLS
- Secure authentication
- Rate limiting
- Bot protection
- Input sanitization
- Request size limits
- Secure cookies
- Protected admin routes
- Secret management
- Error handling

Never expose:

- Supabase service-role keys
 - API secrets
 - internal errors
 - database credentials
-

58. HIPAA-AWARE ARCHITECTURE

Design the system to minimize PHI.

Do not claim:

"This website is HIPAA compliant."

Instead, describe it internally as:

HIPAA-aware architecture

If TomLee intends to store PHI, production configuration must satisfy the required HIPAA contractual and technical requirements of the selected infrastructure vendors.

The system must:

- minimize PHI
- use least privilege
- protect sensitive tables
- restrict storage access
- protect admin accounts
- avoid sensitive information in email
- maintain secure server-side operations
- document data handling

Final compliance/legal determination is outside the scope of the software itself.

59. PRIVACY POLICY

Create:

/privacy

Include appropriate sections for:

- Information collected
- How information is used
- Contact/inquiry forms
- Cookies
- Analytics
- Third-party services
- Data retention
- User rights where applicable
- Contact information

Do not present generated legal text as legal advice.

Mark legal review as required.

60. TERMS OF USE

Create:

```
/terms
```

Include appropriate general website terms.

Again, require legal review before final publication.

61. SEO

Implement comprehensive SEO.

Every page should have:

- Title
- Meta description
- Canonical
- OpenGraph
- Appropriate structured data
- Semantic headings

Create:

```
/sitemap.xml  
/robots.txt
```

Use clean URLs.

62. LOCAL SEO

TomLee is a Georgia-based business.

Structure the website for local SEO.

Potential content areas:

- Georgia home care
- private-pay home care
- companion care
- personal care
- senior home care
- respite care

- dementia support

Do not create city/service pages for locations the company does not actually serve.

Do not keyword stuff.

Use the actual approved service area.

63. STRUCTURED DATA

Where factually appropriate implement:

- Organization
- LocalBusiness
- Service
- Article
- BreadcrumbList
- FAQPage

Do not fabricate:

- ratings
 - reviews
 - opening hours
 - addresses
 - awards
-

64. ACCESSIBILITY

Target WCAG 2.2 AA principles.

Implement:

- semantic HTML
- keyboard navigation
- visible focus
- proper form labels
- accessible error messages
- screen reader support
- sufficient contrast
- accessible navigation
- accessible accordions
- reduced-motion support

Never rely solely on color.

65. RESPONSIVE DESIGN

Design mobile-first.

Test:

320px
375px
390px
414px
768px
1024px
1280px
1440px
1920px

Ensure:

- no horizontal overflow
- readable typography
- touch-friendly buttons
- responsive navigation
- responsive forms
- responsive tables

66. VISUAL DESIGN

The website should have:

- Premium healthcare aesthetic
- Generous whitespace
- Strong typography
- Soft rounded cards
- Subtle shadows
- Elegant transitions
- High-quality photography
- Brand-colored CTAs
- Consistent iconography

Avoid:

- excessive gradients
- excessive glassmorphism
- excessive animation
- neon colors
- crowded cards
- generic template layouts
- overly clinical UI

Use animation only where it improves the experience.

67. IMAGE DIRECTION

Preferred imagery:

- Caregiver and senior
- Caregiver assisting with daily routines
- Family interaction
- Companion care
- Warm home environment
- Genuine smiles
- Diverse families
- Professional caregivers

Avoid:

- hospital rooms
- emergency medical scenes
- doctors unless relevant
- unrealistic stock photography
- imagery implying services TomLee does not provide

Use Next.js image optimization.

68. TYPOGRAPHY

Use a modern, highly readable font system.

Headings:

Strong but friendly.

Body:

Highly readable.

Do not use overly decorative healthcare fonts.

Use consistent:

- H1
- H2
- H3
- body
- small text
- buttons
- labels

69. DESIGN SYSTEM COMPONENTS

Create reusable components:

```
Header
Footer
Container
Button
SectionHeading
Hero
ServiceCard
ServiceGrid
ResourceCard
ResourceGrid
FAQAccordion
CTASection
TrustBadge
FeatureCard
Testimonial
ContactForm
CareInquiryForm
FormStep
ProgressIndicator
FormSummary
AdminSidebar
AdminHeader
DataTable
StatusBadge
```

Modal
Toast
EmptyState
LoadingState
ErrorState

Do not duplicate UI unnecessarily.

70. NEXT.JS ARCHITECTURE

Use:

- Next.js App Router
- TypeScript
- Server Components by default
- Client Components only when necessary
- Server Actions/API routes for sensitive operations
- Dynamic routes
- Metadata API
- Image optimization

Recommended structure:

```
app/  
  (public)/  
    page.tsx  
    about/  
    services/  
    resources/  
    contact/  
    request-care/  
    privacy/  
    terms/  
  
  admin/  
    page.tsx  
    leads/  
    services/  
    resources/  
    faqs/  
    qualification-rules/  
    settings/  
    users/
```



```
api/

components/
  ui/
  layout/
  public/
  forms/
  admin/

lib/
  supabase/
  auth/
  validation/
  qualification/
  email/
  security/
  seo/

types/

supabase/
  migrations/
  seed/

public/
```

71. VALIDATION

Use Zod or equivalent robust schema validation.

Validate:

- forms
- admin inputs
- database operations
- query parameters
- route parameters

Never trust client-side validation alone.

72. ERROR HANDLING

Implement:

- loading states
- empty states
- error states
- form errors
- database errors
- authentication errors
- 404
- submission success
- submission failure

Do not expose technical errors to users.

73. NO FAKE FUNCTIONALITY

Every button must:

- work
- navigate
- submit
- trigger an action

or be clearly disabled.

Do not create:

- fake dashboards
 - fake analytics
 - fake testimonials
 - fake reviews
 - fake contact details
 - fake service statistics
-

74. CONTENT PLACEHOLDERS

Where client information is missing, use:

[CLIENT TO PROVIDE]

Do not use lorem ipsum in final content.

Do not invent company history.

Do not invent service areas.

Do not invent contact information.

75. COPYWRITING STYLE

The copy should be:

- Warm
- Reassuring
- Professional
- Human
- Clear
- Concise
- Family-focused
- Trust-building

Avoid:

- fear-based marketing
- medical jargon
- exaggerated promises
- corporate language
- unnecessary long paragraphs

The visitor should feel:

"These people understand what my family is going through."

76. CONVERSION STRATEGY

Primary CTA:

REQUEST CARE

Secondary CTAs:

GET A FREE CARE ASSESSMENT

EXPLORE SERVICES

TALK TO TOMLEE

Every major page should have a logical conversion path.

77. ANALYTICS

Prepare for analytics.

Track non-sensitive events such as:

```
page_view
request_care_started
request_care_completed
cta_clicked
service_viewed
resource_viewed
switching_provider_selected
```

NEVER send PHI or sensitive form contents to analytics platforms.

78. PERFORMANCE

Optimize for Core Web Vitals.

Use:

- optimized images
- lazy loading
- responsive images
- minimal JavaScript
- server rendering
- code splitting
- optimized fonts
- caching where appropriate

Avoid unnecessary third-party scripts.

79. TESTING

Create tests for:

Qualification engine

- Private Pay
- Medicaid
- Waiver
- Approved service
- Unsupported service
- Service area
- urgency
- switching-provider lead

Forms

- valid
- invalid
- incomplete
- duplicate
- spam

Security

- unauthorized access
- role restrictions
- RLS
- direct API access

Admin

- service CRUD
- resource CRUD
- FAQ CRUD
- qualification rules

80. SECURITY AUDIT

Before completion inspect the entire project for:

- exposed secrets
- service-role key exposure
- broken RLS
- insecure API routes

- unauthorized admin access
- unsafe queries
- XSS
- injection vulnerabilities
- sensitive logging
- public storage buckets
- insecure uploads
- client-only authorization
- excessive data collection

Fix all critical issues.

81. DOCUMENTATION

Create:

```
README.md

docs/
  architecture.md
  database.md
  deployment.md
  security.md
  seo.md
  admin-guide.md
  handover.md
```

README must include:

- Project overview
 - Tech stack
 - Installation
 - Environment variables
 - Supabase setup
 - Database migrations
 - Authentication
 - Local development
 - Testing
 - Deployment
-

82. ENVIRONMENT VARIABLES

Create:

```
.env.example
```

Document all required variables.

Never commit real secrets.

Clearly separate:

```
NEXT_PUBLIC_*
```

from server-only secrets.

83. GIT WORKFLOW

Use meaningful commits.

Example:

```
feat: initialize Next.js application
feat: add TomLee design system
feat: build homepage
feat: add services CMS
feat: add care inquiry flow
feat: add qualification engine
feat: add admin dashboard
fix: secure lead RLS policies
perf: optimize homepage images
```

Do not make one enormous commit containing the entire application.

84. DEVELOPMENT METHODOLOGY

IMPORTANT:

DO NOT attempt to build the entire application in one uncontrolled generation.

Work phase-by-phase.

For every phase:

1. Inspect the existing project.
2. Understand current implementation.
3. Create an implementation plan.
4. Implement.
5. Run TypeScript checks.
6. Run lint.
7. Run tests.
8. Inspect the UI.
9. Fix issues.
10. Verify functionality.
11. Document important decisions.
12. Move to the next phase.

Never overwrite working code unnecessarily.

85. BUILD PHASES

Execute in this exact order.

PHASE 1 — FOUNDATION

Set up:

- Next.js
 - TypeScript
 - Tailwind
 - Supabase
 - ESLint
 - formatting
 - project structure
 - environment configuration
 - Git
-

PHASE 2 — BRAND & DESIGN SYSTEM

Implement:

- TomLee colors
 - typography
 - spacing
 - buttons
 - cards
 - forms
 - navigation
 - logo
 - responsive system
-

PHASE 3 — PUBLIC SITE SHELL

Build:

- Header
 - Footer
 - Global navigation
 - responsive layout
 - CTA system
-

PHASE 4 — HOMEPAGE

Build the full conversion-focused homepage.

Priority:

1. Hero
 2. Switching section
 3. Differentiators
 4. Services
 5. Care philosophy
 6. How it works
 7. Resources
 8. Final CTA
-

PHASE 5 — ABOUT

Build About page.

PHASE 6 — SERVICES CMS

Build:

- Supabase service table
- RLS
- Service admin
- Services listing
- Service detail pages
- Dynamic SEO

PHASE 7 — RESOURCE CENTER

Build:

- Resource database
- Resource admin
- Resource listing
- Resource detail
- Categories
- SEO

PHASE 8 — FAQ

Build database-driven FAQ system.

PHASE 9 — CONTACT

Build contact page and form.

PHASE 10 — REQUEST CARE

Build multi-step inquiry form.

PHASE 11 — LEAD DATABASE

Create:

- lead table
 - validation
 - secure insertion
 - consent
 - data minimization
-

PHASE 12 — QUALIFICATION ENGINE

Build configurable scoring engine.

PHASE 13 — LEAD ROUTING

Implement:

- qualified
- review
- not currently qualified

routing.

PHASE 14 — ADMIN AUTHENTICATION

Build:

- login
 - logout
 - password reset
 - roles
 - protected routes
-

PHASE 15 — ADMIN DASHBOARD

Build:

- overview
- leads
- services

- resources
 - FAQs
 - qualification rules
 - settings
-

PHASE 16 — SECURITY

Perform complete security hardening.

PHASE 17 — SEO

Implement:

- metadata
 - sitemap
 - robots
 - schema
 - OpenGraph
 - canonical URLs
-

PHASE 18 — ACCESSIBILITY

Perform accessibility review.

PHASE 19 — PERFORMANCE

Perform performance optimization.

PHASE 20 — QA & DEPLOYMENT

Perform:

- device testing
- browser testing
- form testing
- admin testing
- security testing
- SEO testing
- production build

- deployment
 - final documentation
-

86. FIVE-DAY DELIVERY TARGET

The original client target is:

5 business days plus client review cycles.

Structure the first production sprint approximately as:

DAY 1

- Project initialization
- Brand system
- Logo
- Colors
- Typography
- Architecture
- Supabase setup

DAY 2

- Public shell
- Homepage
- About
- Services

DAY 3

- Resources
- FAQ
- Contact
- Request Care
- Database

DAY 4

- Lead qualification
- Admin dashboard
- Authentication
- Security
- Compliance pages

DAY 5

- QA
- Responsive testing
- SEO
- Performance
- Deployment
- Documentation

Client review should run in parallel wherever possible.

87. CLIENT REVIEW ITEMS

Prepare the application so the client can easily review:

Brand

- Logo
- Colors
- Typography

Content

- Company description
- Services
- Service descriptions
- About copy
- Contact details

Claims

- Free assessment
- Same-day/next-day start
- caregiver matching
- backup guarantee
- family updates
- fall-safety check
- CNA/CMA experience
- flexible care
- no contracts

Business

- Service area
- Payment methods
- Lead qualification rules

- Notification email
-

88. HANDOVER

The final handover must include:

- Source code
- Git repository
- Supabase project
- Hosting
- Domain
- DNS
- SSL
- Email
- Admin account
- Brand assets
- Documentation
- Environment variable guide
- Database documentation
- Deployment guide
- Admin guide
- Security documentation

No critical system component should depend permanently on the developer's personal account.

89. FINAL DEFINITION OF DONE

The application is NOT complete until all of the following are verified:

- [] Homepage complete
- [] About complete
- [] Services complete
- [] Dynamic service pages work
- [] Future service architecture works
- [] Resources complete
- [] Resource detail works
- [] FAQ works
- [] Contact works
- [] Request Care works
- [] Lead qualification works
- [] Switching-provider leads are identifiable
- [] Lead routing works
- [] Admin authentication works

- ☐ Admin roles work
- ☐ Admin services work
- ☐ Admin resources work
- ☐ Admin FAQs work
- ☐ Qualification rules work
- ☐ Admin lead management works
- ☐ RLS enabled
- ☐ No exposed secrets
- ☐ Server validation implemented
- ☐ Spam protection implemented
- ☐ Privacy Policy exists
- ☐ Terms exist
- ☐ SEO implemented
- ☐ Sitemap implemented
- ☐ Robots implemented
- ☐ Schema implemented where appropriate
- ☐ Accessibility reviewed
- ☐ Mobile responsive
- ☐ Desktop responsive
- ☐ Performance reviewed
- ☐ Error handling implemented
- ☐ 404 implemented
- ☐ Production build succeeds
- ☐ TypeScript passes
- ☐ Lint passes
- ☐ Critical tests pass
- ☐ Security review completed
- ☐ Documentation completed
- ☐ Handover documentation completed

90. NON-NEGOTIABLE RULES

You MUST NOT:

1. Invent business information.
2. Invent testimonials.
3. Invent certifications.
4. Invent licenses.
5. Invent awards.
6. Invent service areas.
7. Claim Medicaid participation.
8. Claim waiver participation.
9. Claim HIPAA compliance without appropriate organizational/vendor requirements.
10. Expose secrets.
11. Expose Supabase service-role credentials.

12. Store unnecessary medical information.
 13. Send sensitive information unnecessarily through email.
 14. Hard-code future services.
 15. Hard-code qualification rules.
 16. Build fake functionality.
 17. Create inaccessible forms.
 18. Ignore mobile responsiveness.
 19. Use generic lorem ipsum in the finished application.
 20. Treat the public website as the entire product.
-

91. MOST IMPORTANT PRODUCT PRINCIPLE

The visitor should immediately understand:

TOMLEE HELPS FAMILIES GET RELIABLE HOME CARE.

And a visitor who is unhappy with another agency should immediately think:

"Maybe TomLee can do better."

The site should answer these questions quickly:

Can you help my loved one?

What services do you provide?

Can I trust your caregivers?

How quickly can care start?

How much flexibility do I have?

What happens if my current caregiver doesn't show up?

Will my family be kept informed?

Can I switch from another agency?

How do I get started?

The website should remove uncertainty and make the next step obvious.

92. FINAL ANTIGRAVITY COMMAND

Build this as a production-quality application.

Do not build a static mockup.

Do not generate the entire project blindly in one operation.

Start with **PHASE 1 — FOUNDATION**.

Before coding, inspect the project environment and create a concise implementation plan.

Then implement Phase 1 only.

After Phase 1:

- verify the project
- run checks
- fix issues
- document what was created

Then wait for the next phase instruction.

For every subsequent phase, follow the same controlled development process.

The final result must be a premium, conversion-focused, secure, responsive, SEO-friendly and maintainable home-care platform for TomLee Homecare LLC built with Next.js and Supabase.